



Sass & BEM

Módulo 7



Elaborado por: Jesua Luján

Jesua Hadai Luján



DEV.F.:



DEV.F.

● Módulo 3: Integrando Sass & BEM



🎯 *Objetivo: Aprender a utilizar Sass con la metodología BEM (Bloques, Elementos y Modificadores)*



Contenido:



Introducción a **Sass y BEM**. 🤔



Aprender qué es la **metodología BEM**. 🧑



Mejores prácticas para **escribir código Sass usando BEM**. 🧑



Aplicando BEM en Sass 🐱



React



Preprocesadores

preprocesadores de
CSS



DEV.F.:



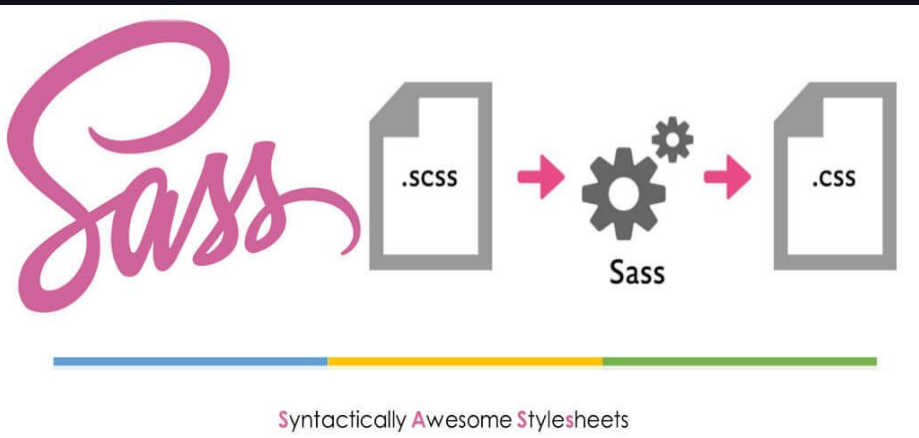
✨ Preprocesadores CSS

Los preprocesadores de CSS son herramientas que extienden el CSS estándar y permiten trabajar de manera más eficiente y organizada. Básicamente, **te ofrecen funcionalidades que el CSS tradicional no tiene**, y las convierten en código CSS que los navegadores pueden entender.



✨ ¿Qué es un preprocesador de CSS?

Un preprocesador **es un lenguaje que extiende las capacidades de CSS** y permite escribir hojas de estilo de una manera más dinámica y estructurada. Los preprocesadores son compilados en CSS estándar antes de que los navegadores los procesen.





✨ ¿Qué es un preprocesador de CSS?

Este pseudocódigo está formado por **variables, condicionales, bucles o funciones**, elementos habituales de cualquier lenguaje de programación.

Por este motivo, podríamos decir que tenemos **un lenguaje de programación cuya misión es la de generar el código CSS**





Sass

```
$linkcolour: #f60;  
  
a {  
  color: $linkcolour;  
}  
  
.headers {  
  color: $linkcolour;  
}
```



P
R
E
P
R
O
C
E
S
S
O
R

Output - CSS

```
a {  
  color: #f60;  
}  
  
.headers {  
  color: #f60;  
}
```



```
@linkcolour: #f60;  
  
a {  
  color: @linkcolour;  
}  
  
.headers {  
  color: @linkcolour;  
}
```



Beneficios de Preprocesadores CSS

Nuevas funcionalidades: Permiten realizar acciones que no son posibles en CSS tradicionales, como lógica condicional.

Reutilización: Favorecen reusar mucho del código repetitivo de CSS, ahorrando tiempo y esfuerzo.

Cambios rápidos: Al solo cambiar valor de variables o hacer uso de mixins.

Compatibilidad: Facilita la creación de código compatible con múltiples navegadores.

Sass

SASS

preprocesador de CSS



DEV.F.



Sass



¿Que es Sass?

Sass (**Syntactically Awesome Stylesheets**) es un lenguaje de hoja de estilos en cascada.

Sass es un procesador CSS, **ojo no solo es exclusivo de javascript** por lo que es una herramienta que nos permite generar, de manera automática, hojas de estilo, añadiéndoles características que no tiene CSS, y que son propias de los lenguajes de programación, como pueden ser **variables, funciones, selectores anidados, herencia, etcétera.**



Sass



¿Que es Sass?

Sass es uno de los preprocesadores más populares y ofrece una gran cantidad de características útiles para organizar tu CSS. Existen dos sintaxis para Sass:

- **.scss:** La sintaxis más común, similar al CSS tradicional, pero con la capacidad de usar variables, anidamiento, mixins, etc.
- **.sass:** Sintaxis más compacta, sin llaves ni punto y coma.



[Playground](#)

[Install](#)

[Learn Sass](#)

[Blog](#)

[Documentation](#)

[Get Involved](#)

CSS with superpowers

Sass is the most mature, stable, and powerful professional grade CSS extension language in the world.



Current Releases: [Dart Sass 1.89.2](#) [LibSass 3.6.6](#) [Ruby Sass](#)  [Implementation Guide](#)



Playground

[Copy URL](#)

SCSS Sass v1.89.2

```
1 @use "sass:list";
2 @use "sass:color";
3
4 $font-stack: Helvetica, Arial;
5 $primary-color: #333;
6
7 body {
8   $font-stack: list.append($font-stack, sans-serif);
9   font: $font-stack;
10 }
11
12 a {
13   color: $primary-color;
14
15   &:hover{
16     color: color.scale($primary-color, $lightness: 20%);
17   }
18 }
19
20 @debug $font-stack;
```

Expanded Compressed CSS (Compiled)

```
1 body {
2   font: Helvetica, Arial, sans-serif;
3 }
4
5 a {
6   color: #333;
7 }
8 a:hover {
9   color: rgb(91.8, 91.8, 91.8);
10 }
```

Console

@debug:20 Helvetica, Arial

<https://sass-lang.com/playground/>

Sass © 2006–2025 the Sass team, and numerous contributors. It is available for use and modification under the MIT License.

[Sass on GitHub](#) [Website Source Code](#) [Style Guide](#) [Community Guidelines](#)

[@sass@front-end.social](#)



SCSS

dart-sass v1.32.12

```
1 // SCSS
2 $myFont: Helvetica, sans-serif;
3 $myFontColor: red;
4 $myFontSize: 18px;
5 $myWidth: 680px;
6
7 body {
8   font-family: $myFont;
9   font-size: $myFontSize;
10  color: $myFontColor;
11 }
12
13 #container {
14   width: $myWidth;
15 }
```

CSS

```
1 body {
2   font-family: Helvetica, sans-serif;
3   font-size: 18px;
4   color: red;
5 }
6
7 #container {
8   width: 680px;
9 }
10
```

Para crear variables usamos el \$ para nombrarlas, posteriormente definiremos su valor.

Para usar variables, solo escribiremos el \$ seguido del nombre de la variable.



Anidamiento de Reglas en Sass (Nesting)

```
SCSS                                     dart-sass v1.32.12
1  // SCSS
2  nav {
3    ul{
4      margin: 0;
5      padding: 0;
6      list-style: none;
7    }
8    li {
9      display: inline-block;
10   }
11   a {
12     display: block;
13     padding: 6px 12px;
14     text-decoration: none;
15   }
16 }
17
```

```
CSS
1  nav ul {
2    margin: 0;
3    padding: 0;
4    list-style: none;
5  }
6  nav li {
7    display: inline-block;
8  }
9  nav a {
10   display: block;
11   padding: 6px 12px;
12   text-decoration: none;
13 }
14
```



Si colocamos elementos dentro de las `{ }` de otro elemento padre, estas serán tomadas en cuenta. Por lo que resulta cómodo para agrupar estilos que tengan que ver con un elemento en concreto.



Anidamiento de Propiedades

DEV.E.

SCSS

dart-sass v1.32.12

```
1 // SCSS
2 body {
3   font:{
4     family: Helvetica, sans-serif;
5     size: 18px;
6     weight: bold;
7   }
8 }
9
10 .title {
11   text:{
12     align: center;
13     transform: lowercase;
14     overflow: hidden;
15   }
16 }
17
```

CSS

```
1 body {
2   font-family: Helvetica, sans-serif;
3   font-size: 18px;
4   font-weight: bold;
5 }
6
7 .title {
8   text-align: center;
9   text-transform: lowercase;
10  text-overflow: hidden;
11 }
```





Importar Archivos Parciales con Sass (_Partials)

DEV.E.

```
SCSS dart-sass v1.32.12
1 // SCSS
2 // _colors.scss
3 $myRed: #b75038;
4 $myBlue: #1376b8;
5 $myGreen: #378f66;
6
```

```
SCSS dart-sass v1.32.12
1 // SCSS
2 // main.scss
3 x @use 'colors';
4
5 body {
6   font-family: Helvetica, sans-serif;
7   font-size: 18px;
8   color: $myBlue;
9 }
10
```

```
CSS
1 body {
2   font-family: Helvetica, sans-serif;
3   font-size: 18px;
4   color: #1376b8;
5 }
6
```

Puedes crear archivos Sass parciales que contengan pequeños fragmentos de CSS que puedes incluir en otros archivos Sass.

Un parcial es un archivo Sass nombrado con un guión bajo inicial (ejemplo. `_colors.scss`).

El guión bajo permite a Sass saber que el archivo es sólo un archivo parcial y que no debe ser generado en un archivo CSS.

Los parciales de Sass se utilizan con la regla `@use`.





Reutilización de estilos: @mixins en Sass

SCSS dart-sass v1.32.12

```
1 // SCSS
2 @mixin important-text {
3     color: white;
4     font-size: 32px;
5     font-weight: bold;
6 }
7
8 .danger {
9     @include important-text;
10    background-color: red;
11 }
12
13 .notice {
14     @include important-text;
15     background-color: blue;
16 }
17
```

CSS

```
1 .danger {
2     color: white;
3     font-size: 32px;
4     font-weight: bold;
5     background-color: red;
6 }
7
8 .notice {
9     color: white;
10    font-size: 32px;
11    font-weight: bold;
12    background-color: blue;
13 }
14
```

Los mixins nos ayudan a crear bloques de estilos reutilizables.

Se declaran con **@mixin nombre-del-mixin**

Para usarlos, utilizamos **@include nombre-del-mixin**



🧠 Los @mixins con parámetros y valores por defecto

SCSS

dart-sass v1.32.12

```
1 // SCSS
2 // Mixins con dos argumentos
3 @mixin bordered($color, $width) {
4     border: $width solid $color;
5 }
6
7 .myArticle {
8     @include bordered(blue, 1px)
9 }
10
11 .myNotes {
12     @include bordered(red, 2px)
13 }
14
15 // Mixins con valores por defecto
16 @mixin with-border($color: blue, $width: 1px) {
17     border: $width solid $color;
18 }
19
20 .myTips {
21     @include with-border($color: orange);
22 }
```

CSS

```
1 .myArticle {
2     border: 1px solid blue;
3 }
4
5 .myNotes {
6     border: 2px solid red;
7 }
8
9 .myTips {
10     border: 1px solid orange;
11 }
12
```



🧠 Los @mixins + if

SCSS Sass

```
@mixin avatar($size, $circle: false) {  
  width: $size;  
  height: $size;  
  
  @if $circle {  
    border-radius: $size / 2;  
  }  
}  
  
.square-av {  
  @include avatar(100px, $circle: false);  
}  
  
.circle-av {  
  @include avatar(100px, $circle: true);  
}
```

CSS

```
.square-av {  
  width: 100px;  
  height: 100px;  
}  
  
.circle-av {  
  width: 100px;  
  height: 100px;  
  border-radius: 50px;  
}
```

[Sass: @if and @else \(sass-lang.com\)](https://sass-lang.com)





Sass v1.8.22

Syler



1,470,171

★★★★★ (7)

Indented Sass syntax Highlighting, Autocomplete & Formatter

Disable



Uninstall



This extension is enabled globally.

Details

Feature Contributions

Changelog

Runtime Status

build passing

codefactor A

version v1.8.22

downloads 6.2M

installs 1.5M

last commit july

issues 11 open

![Tested With](https://img.shields.io/badge/Syntax%20Tested%20With-test--

grammar-red?style=for-the-badge)

USING

SASS-FORMATTER

*Indented Sass syntax highlighting,
autocomplete & Formatter for
VSCode*

Categories

Programming
Languages

Formatters

Extension
Resources

Marketplace
Repository
License



BEM

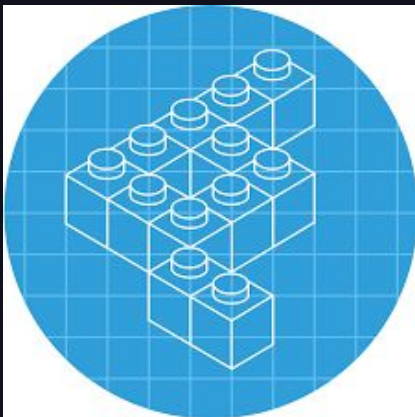
**(Bloque Elemento
Modificador)**



DEV.F.:



Block **E**lement **M**odifier



¿Qué es BEM? 🤔

BEM (Block, Element, Modifier) es una metodología de nomenclatura y organización de clases CSS que ayuda a estructurar el código de manera **clara, reutilizable y mantenible**. Ideal para proyectos grandes y equipos de desarrollo 👨‍💻👩‍💻, porque mejora la **colaboración** y facilita la **comprensión del código**.

Es decir, es una manera de nombrar las clases de los nodos de tu HTML para posteriormente **seleccionarlos con CSS de una manera fácil, sencilla y clara**.



Componentes de BEM

BEM se desglosa en tres partes principales:

1. **Block (Bloque)** 🏢: Es el **componente** o parte independiente de la interfaz. Es una unidad autónoma que tiene un propósito específico y puede ser reutilizada en otros contextos.

Ejemplo: Un bloque puede ser un header, un button, una card, etc.

2. **Element (Elemento)** 🔑: Son las partes **internas** del bloque. Los elementos no tienen sentido por sí mismos sin el bloque al que pertenecen.

Ejemplo: Un botón dentro de una barra de navegación (`.nav__button`), un título dentro de una tarjeta (`.card__title`).

3. **Modifier (Modificador)** 🎨: Son las **variaciones** de bloques o elementos. Permiten cambiar su apariencia, comportamiento o estado. Los modificadores describen cómo un bloque o elemento se ve o se comporta bajo ciertas condiciones.



`.bloque__elemento--modificador{}`

`.bloque`
`.bloque__elemento`
`.bloque--modificador`
`.elemento--modificador`
`.bloque__elemento--modificador`

*Si un nombre tiene requiere “un espacio”
usamos un guión simple como separador,
otras alternativas es usar notación
camelCase o un guión bajo.*

<https://en.bem.info/methodology/naming-convention/>



Componentes de BEM

BEM son tres siglas:

- B de bloque.
- E de elemento.
- M de modificador.

BEM nos indica qué debemos usar clases para nuestros selectores con una estructura definida (bloque, elemento, modificador).

BEM tiene unas convenciones acerca de cómo debemos nombrar a nuestras clases de CSS.

.bloque__elemento--modificador

Bloque

```
HTML (bloque)  
  
<header class="header">  
  <!-- aquí van mis elementos -->  
</header>
```

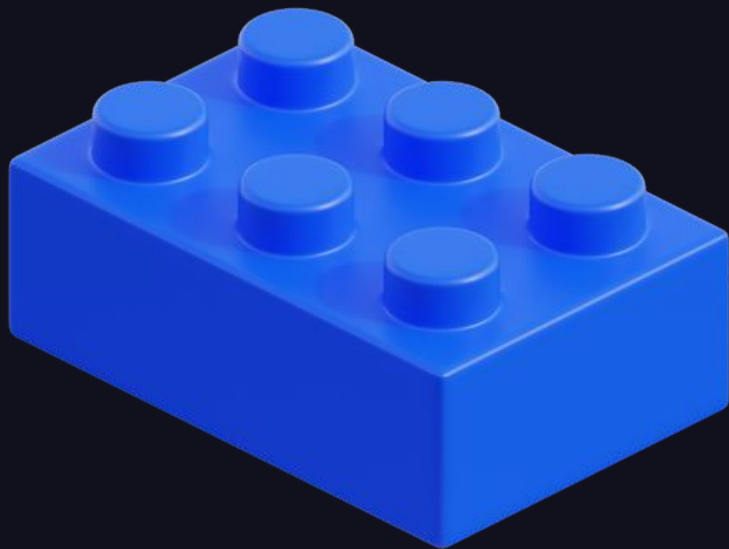
Un bloque es una sección independiente que tiene significado propio por sí solo.

Contiene todos los nodos HTML de una estructura a la que te estés refiriendo.

Para aclarar esto las siguientes etiquetas HTML son consideradas bloques a simple vista: **header, sidebar, aside, main, footer, article, section, ul, ol, div.**



`.bloque__elemento--modificador`



Elemento

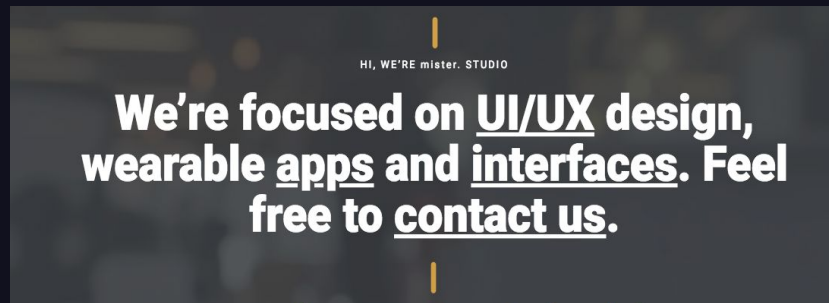
Un elemento **son porciones más pequeñas internas a un bloque.**

El elemento es una pieza de un bloque. El bloque es el todo y los elementos son las piezas.

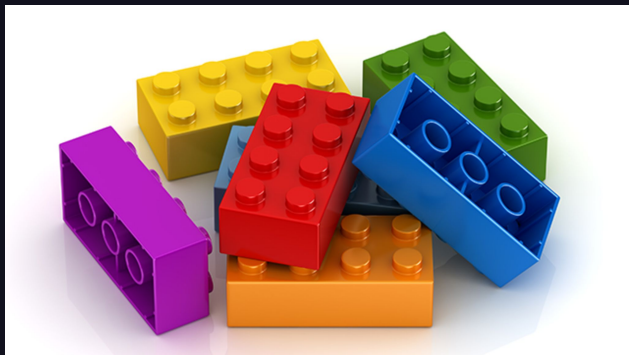
El método BEM nos indica que cada elemento nace a partir de la clase del bloque que conforma seguida de dos guiones bajos y el nombre clave del elemento.

`.bloque__elemento--modificador`

`.hero__text--small`



`.hero__text--big`



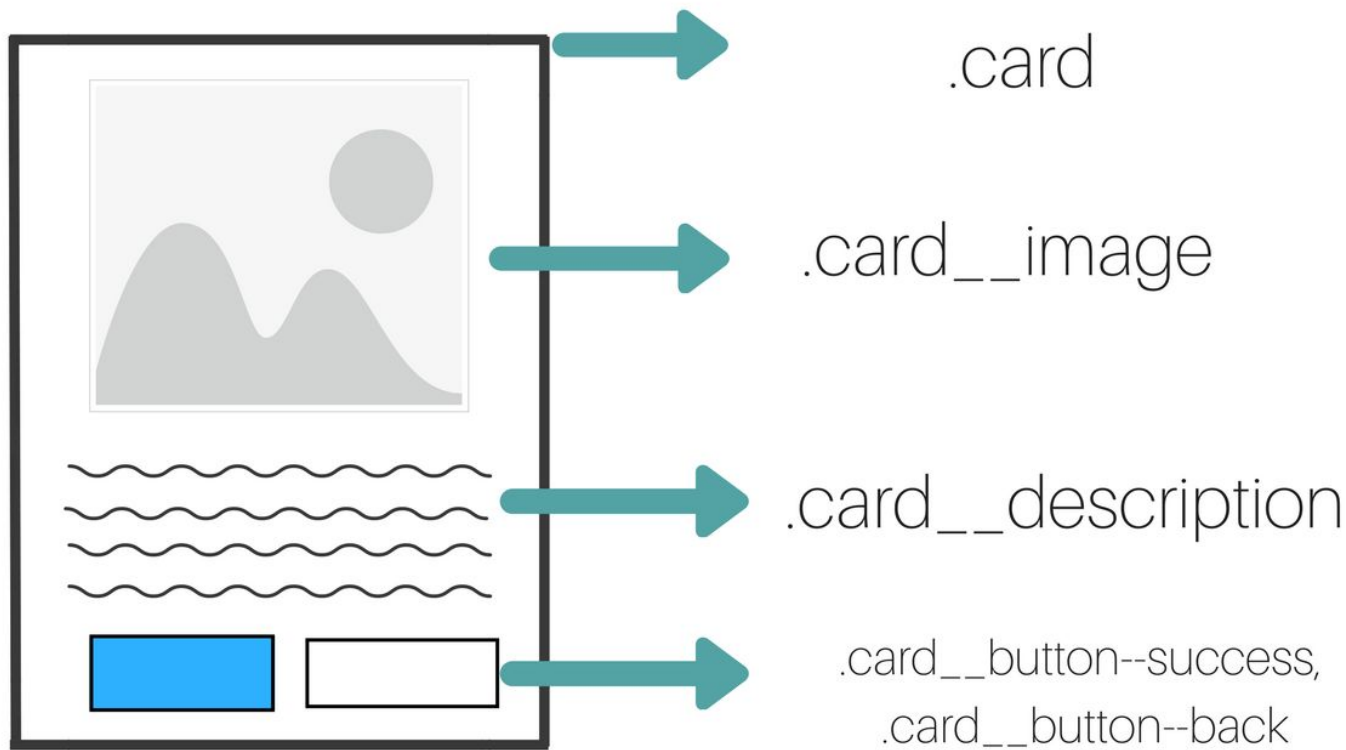
Modificador

Para modificar el estilo de un elemento específico, existen los modificadores.

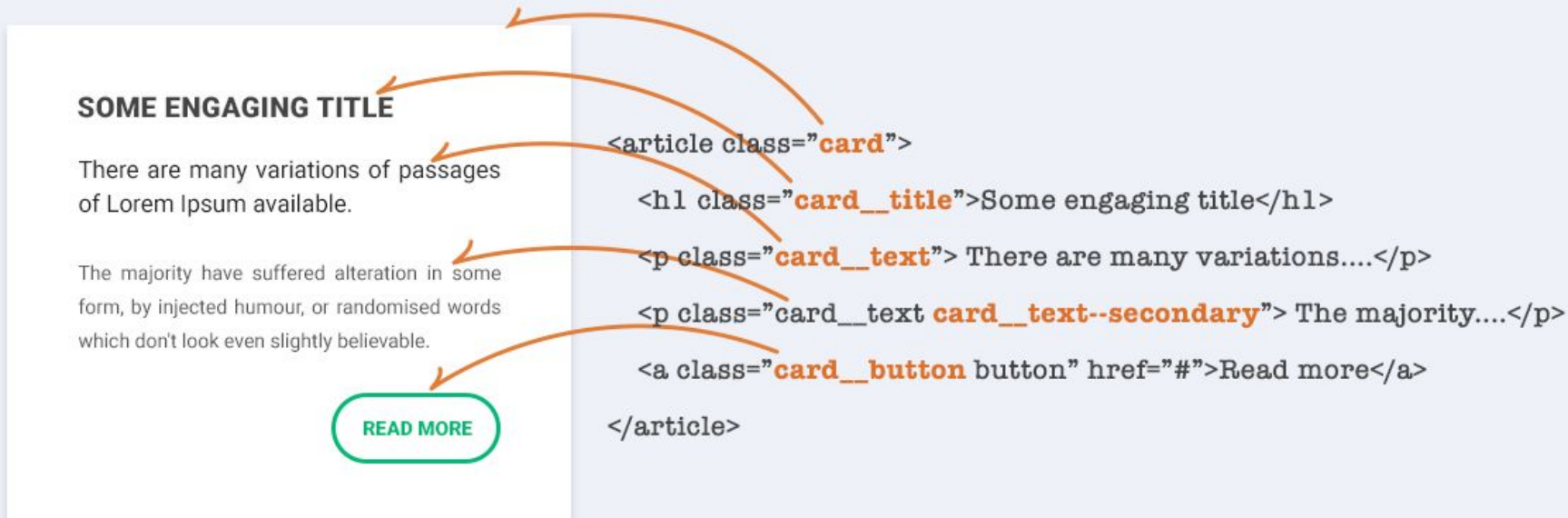
Estos se usan agregando un doble guión justo después del elemento o bloque que se quiere modificar.

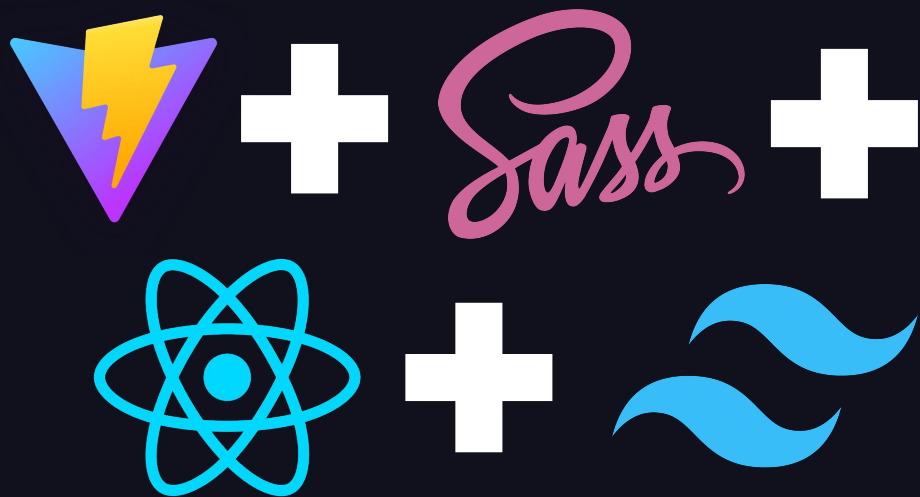
Un modificador nos permite **tener variantes de estilo sobre un bloque o elemento.**

Ejemplo de nomenclatura BEM aplicada



Ejemplo de BEM aplicado en HTML





Stack tecnológico



DEV.F.:

Para más información, pueden consultar:

- <https://sass-lang.com/guide>
- <https://platzi.com/blog/bem/>
- <https://9elements.com/bem-cheat-sheet/>
- <https://medium.com/williambastidasblog/metodologías-o-arquitecturas-css-oocss-bem-smacss-itcss-atomic-design-a1a3cfbfa6c9>

**el conocimiento es
poder**

